

Janus4K AI Core 微架构规格书

版本: v0.4-draft

日期: 2026-04-26

状态: AS 收敛稿, 已合入用户确认设计点

来源依据: Janus4k.drawio / Overview 页 / Vector Pipe 页

参考模板: AS/TMU/TMU_SPEC.md / AS/Vector/VECTOR_CORE_SPEC.md

1. 概述

1.1 文档状态

- 文档名称: Janus4K AI Core Architecture Spec
- 版本: v0.4-draft
- 日期: 2026-04-26
- 来源依据:
 - Janus4k.drawio
 - Overview 页
 - Vector Pipe 页
- 文档目的: 把图中已经表达出的架构意图和本轮确认的设计约束收敛成一版可评审、可继续落实现的 AS 草案

1.2 证据等级说明

为避免把“图上明确写了什么”和“为了形成规格而做的工程推断”混在一起, 本文使用以下标记:

- [M]: 图示明确。图中直接给出了模块、连接、数值或注释。
- [I]: 结构推断。图没有逐字写明, 但从布局、连线、标注位置和上下文可以较高置信度推断。
- [O]: 开放项。图中或本轮问答仍未给出足够信息, 本文只保留占位定义或需要 RTL 对齐的 TBD。

2. 范围与边界

2.1 本文覆盖范围

本文只覆盖当前图中已有实质内容的部分:

- Overview 页中的顶层数据通路和带宽关系
- Vector Pipe 页中的向量执行前端、读路径、调度结构、双执行流水和依赖处理

2.2 仍需后续细化的部分

本轮已确认 BCC Scalar Pipe、TileReg、Output Buffer、TMA、Cube 的关键语义。下列内容仍保留 TBD 或后续 RTL 对齐:

- BCC 指令编码位宽、descriptor 压缩格式和异常/错误上报
- TileReg 1MB 地址空间到逻辑 bank/slice 的精确编码

- `Output Buffer`、`Global Src Buffer`、各 uOp queue 的物理深度
- `Cube` 阵列规模、LOA/LOB 深度、accumulate/输出 layout
- mask Tile 格式、dtype/packing 编码和不同 element width 的 uOp 拆分规则

2.3 名词边界

更新后的顶层 drawio 已补充 `BCC`、`Cube`、`TMA` 的职责，因此本文：

- 将 `BCC` 定义为 `Block Control Core`
- 将 `Cube` 定义为参考 DaVinci 的矩阵计算单元
- 将 `TMA` 定义为 TileReg 与 DDR/Memory 之间的访存/搬运单元
- 对尚未明确的位宽、队列深度、精确 bank 地址编码和 mask/dtype 编码继续显式标注 `[O]` 或 `TBD`

3. 一句话架构总结

Janus4K 当前呈现出来的不是一个“只有矩阵阵列”的加速块，而是一个以 `Tile Register File` 为数据中心、以 `Block -> TileOp -> uop` 三级组织为控制骨架、以 `BCC Scalar Pipe + Operand Collector + Read Port Arbitration + Global Src Buffer + Output Buffer + Vector/Cube/TMA` 为吞吐保障机制的 tile-centric AI 核内执行子系统。

更具体地说：

- `Tile Register File / TileReg` 是整个 AI 核内共享的数据 buffer，定位类似 DaVinci 的 UB 或 NVIDIA 的 Shared Memory `[M]`
- `BCC` 将计算任务按 Block 粒度分发到 `Vector Core`、`Cube` 或 `TMA`，一个 Block 只能发往一个目标 PE；`BCC` 内含 `GPR` 与 `AB/AM/LSU` 标量 pipe `[M]`
- `Operand Collector` 负责确定源操作数来自哪里：命中 `Output Buffer` 时走 forwarding，未命中时向 `TileReg` 发起 read request `[M][I]`
- `Output Buffer` 对应 drawio 中的 `Dst Buffer`，是 forwarding、write-port arbitration、跨 Pipe forwarding 和 reduction reuse 的关键结构 `[M]`
- 两条执行流水的首要目标是提升实际算子利用率，而不是单纯堆叠峰值算力 `[M]`
- `Read Port Arbitration -> Tile Register File -> Src Buffer` 是当前图上唯一被明确点名为时延/拥塞敏感的关键路径 `[M]`

4. 设计目标

根据图中结构与注释，可以把 Janus4K 这版设计的目标提炼为以下几条：

1. 以 `Tile Register File / TileReg` 为中心统一承载向量执行、`Cube` 协同和 `TMA` 搬运的数据交换。
2. 支持 Block 描述符拆成 `BCC` 指令，至少覆盖 `Bstart`、`B.IOT(3 src + 1 dst)` 和 `TMA TLOAD`。
3. 在多周期执行和多级数据依赖下，通过 `Output Buffer` 和 `Data Forwarding` 尽量减少 bubble。
4. 将“读数是否完成”“依赖是否满足”“执行流水是否可用”这三类问题解耦处理，降低单点阻塞。
5. 通过双执行流水将不同类型算子拆分到不同功能路径，提升实际 compute-unit utilization。

6. 在物理实现层面承认读路径拥塞会显著拉长供数时延，并通过缓冲和调度隐藏这一时延。
7. 在小容量、近执行体的本地数据结构中维持热工作集，避免每次都走更远、更慢的层级。

5. 图示解读后的工作定义

为了把图扩写成 AS，需要先把几个图上没有明文定义、但文中会反复使用的概念工作化。

5.1 Tile 与 Tile slice

- [M] Vector 侧每个逻辑 Tile 为 4 KB；每个 Vector TileOp 的源输入按 4 KB srcTile 建模。
- [M] Cube/TMA 可使用更大的 Tile，容量必须是 4 KB 的整数倍。
- [M] 图中多次出现 512 B，本文将其定义为 tile slice、Output Buffer entry 粒度和 Vector Core 每拍计算/传输粒度。
- [M] Vector 计算单元宽度为 512 B/cycle。例如 TADD 这类向量算子可按 512 B slice 上 pipe，一个 4 KB Tile 需要拆成 8 个 slice。
- [M] 顶层 drawio 中 TileReg 可抽象为 4 个逻辑 bank，每个 bank 每拍读/写 512 B，因此一次对 TileReg 的满带宽交互为 2 KB/cycle。
- [M] 一个 4 KB Tile 在 TileReg 内地址总是连续，按 512 B slice 编号分布到逻辑 bank；各 PE 总能无 bank conflict 地读出某个 Tile 的 2 KB 数据窗口。
- [I] 512 B 是计算和局部 pipe 的最重要粒度；4 KB 是 Vector Tile 级依赖、任务输入输出和 TileReg 分配的逻辑粒度。

5.2 TileOp

- [M] 图中存在 TileOp Issue Queue、Example of intra-TileOp dependence
- [M] 每个 TileOp 的 Vector 源输入是 4 KB srcTile；mask Tile 可作为可选输入。
- [I] 本文将 TileOp 视为前端看到的 tile 级操作单位，其输入输出以 Tile 建立依赖关系，内部可按 opclass、element width 和数据形态展开成多个 uOp 或多个 512 B slice 迭代。

5.3 uOp

- [M] 图中存在独立的 uOp Issue Queue
- [I] 本文将 uOp 定义为可被独立调度到执行流水的最小执行单位

5.4 Forwarding-ready result

- [M] 图中 Dst Buffer 旁明确写有 data forwarding
- [M] 本文将 drawio 中的 Dst Buffer 正式命名为 Output Buffer。执行结果在正式写回 TileReg 之前，可以先驻留在 Output Buffer 中供后继 uOp 旁路使用。

5.5 Global / Local buffer

- [M] Output Buffer 是全局唯一结构。
- [M] Pipe0/Pipe1 之间允许跨 Pipe forwarding。
- [I] drawio 中 Pipe-local Src Buffer / Dst Buffer / Data Forwarding 表达靠近执行入口/出口的局部暂存与前递路径，但架构语义上以全局 Output Buffer 为依赖查询和写回仲裁中心。

6. 顶层架构概览

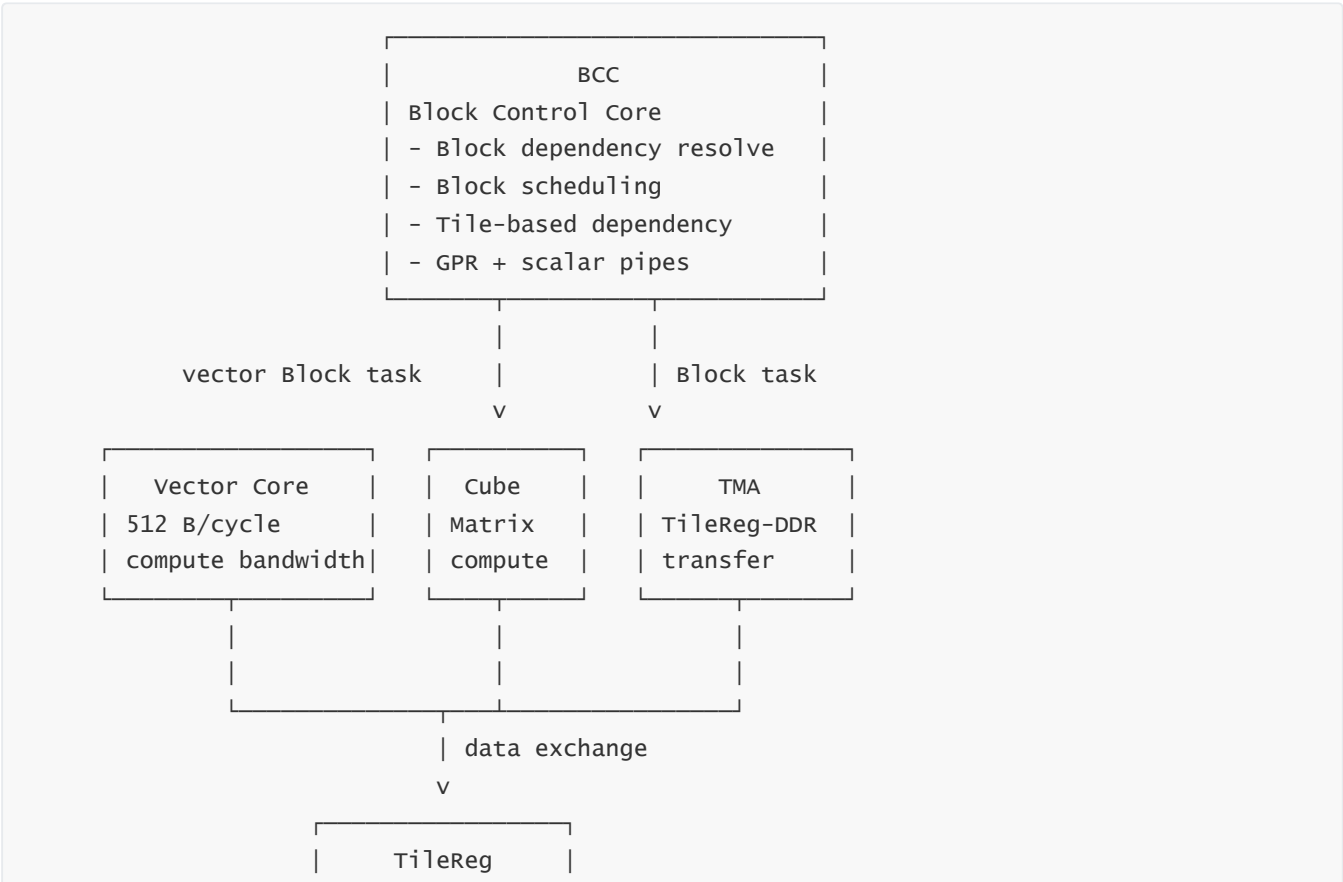
6.1 顶层模块关系

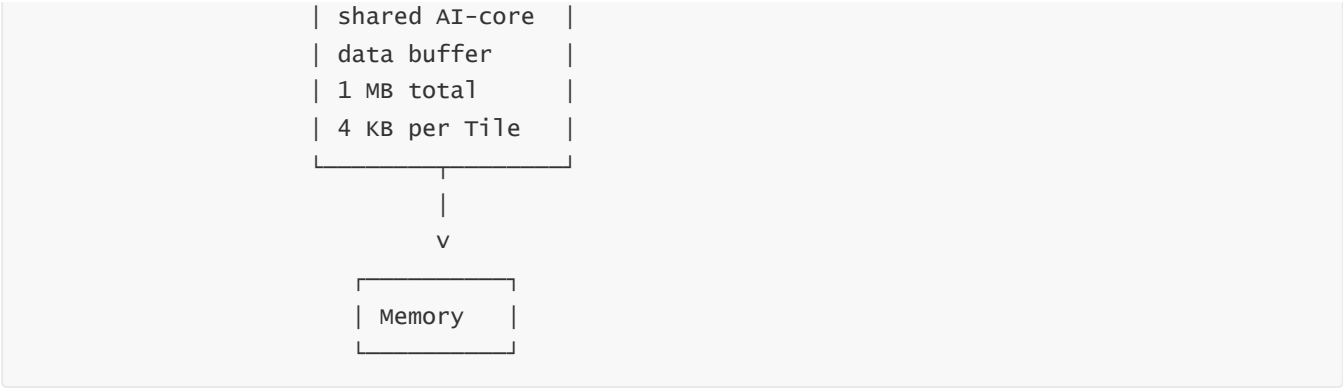
从更新后的 Overview 页可以读出 Janus4K 顶层至少包括以下模块：

- BCC [M]
- Vector Core [M]
- TileReg [M]
- Cube [M]
- TMA [M]
- Memory [M]

这些模块中，TileReg 位于 Vector Core、Cube、TMA 和 Memory 数据交互的中心位置，因此本文将其视为顶层共享数据 buffer。

6.2 顶层系统框图





顶层更新后，BCC 是任务控制入口，负责把一个个 Block 分发到 Vector Core、Cube 或 TMA。Block 的输入输出以 Tile 表示，因此 BCC 可以用 Tile 建立 Block 间依赖关系。

6.3 顶层带宽信息

TileReg 可抽象成 4 个 bank，每个 bank 每拍读出 512B，因此满带宽读为 2KB/cycle。Vector/Cube/TMA 在仲裁后可以连续占用多拍读口。例如 Vector 某个 TileOp 有三个操作数，则可以连续占用三拍读口，依次读出 2KB 的 srcA、2KB 的 srcB、2KB 的 srcC，再把这个 TileOp 拆成 512B uOp/slice，分四次迭代计算完成。

图中可直接提取的数值如下：

项目	图示值	解释
Vector Core 计算带宽	512 B/cycle [M]	Vector 执行单元每拍可处理一个 512 B slice
Tile 逻辑容量	4 KB [M]	每个 Block 的 Tile 输入/输出粒度
TileReg 总容量	1 MB [M]	AI 核内共享数据 buffer 总量
TileReg bank 读带宽	4 x 512 B = 2 KB/cycle [M]	4 个 bank 每拍各读 512 B
Vector TileOp 三源读	srcA/srcB/srcC [M]	一个三源 TileOp 可连续占用 3 拍 TileReg 读口，各读出 2 KB
Vector uOp/slice 粒度	512 B [M]	2 KB 源数据可拆成 4 个 512 B uOp/slice 迭代计算
Cube	矩阵计算单元 [M]	参考 DaVinci 风格 Cube/Tensor compute
TMA	TileReg-DDR 访存 [M]	负责 TileReg 与 DDR/Memory 之间的数据搬运

6.4 顶层方向性观察

- [M] BCC -> Vector Core/Cube/TMA 是计算任务或搬运任务分发路径。
- [M] Vector Core/Cube/TMA <-> TileReg 是数据交互路径。
- [M] TMA 的职责是 TileReg 和 DDR/Memory 之间的数据搬运，因此 TMA <-> TileReg 和 TMA <-> Memory 在架构语义上均为双向数据路径。
- [M] TileReg 是所有计算模块共享的数据 buffer，不是 Vector Core 私有 RF。

6.5 顶层端口利用率观察

- [M] 顶层图强调 数据交互，说明 TileReg 端口不是固定属于某一个计算单元，而是需要在 Vector Core、Cube、TMA 之间仲裁使用。
- [I] 因此 Janus4K 的目标不只是把读写口堆出来，还需要考虑端口复用、共享仲裁和动态占用。

7. 详细模块清单

7.1 模块总表

模块	所在页	作用摘要	证据等级
BCC	Overview / Vector Pipe	Block Control Core, 负责 Block 依赖解析、调度、任务分发和标量 pipe 执行	[M]
GPR	BCC 内部	Block 间标量值交互的通用寄存器文件	[M]
BCC Scalar Pipe	BCC 内部	AB/AM/LSU 标量执行管线	[M]
Vector Core	Overview / Vector Pipe	向量执行单元, 计算带宽 512 B/cycle	[M]
Vector Block	Vector Pipe	Vector Core 内部的 TileOp 前端窗口、源操作数生成与唤醒入口	[M] [I]
TileOp Issue Queue	Vector Pipe	前端 TileOp 级窗口或队列	[M] [I]
Operand Collector	Vector Pipe	聚合源操作数并发起依赖解析	[M]
Output Buffer	Vector Pipe	drawio 中的 Dst Buffer, 负责结果驻留、旁路、写口仲裁和 reduction reuse	[M]
Read Port Arbitration	Vector Pipe	仲裁 Tile Register File 读请求	[M]
Tile Register File / TileReg	Overview / Vector Pipe	AI 核内共享数据 buffer, 总容量 1MB, 每 Tile 4KB	[M]
Global Src Buffer	Vector Pipe	靠近 Tile Register File 的共享读数据缓冲	[M] [I]
uOp Issue Queue	Vector Pipe	可执行 uOp 调度中心	[M]
FMLA/FCVT Pipe	Vector Pipe	FMLA / FCVT 类执行路径	[M]
IALU/PERM/MAC Pipe	Vector Pipe	IALU / PERM / MAC 类执行路径	[M]
SFU	Vector Pipe	EXP / DIV 等特殊函数单元, 计算带宽 256 B/cycle	[M]

模块	所在页	作用摘要	证据等级
Pipe-local Src/Dst/Forward	Vector Pipe	每条 Pipe 本地的输入/结果/旁路结构	[M] [I]
Cube	Overview / Vector Pipe	矩阵计算单元，参考 DaVinci 风格 Cube	[M]
TMA	Overview / Vector Pipe	访存/搬运单元，负责 TileReg 与 DDR/Memory 交互	[M]

7.2 BCC

7.2.1 位置与角色

- [M] BCC 全称为 Block Control Core。
- [M] 更新后的顶层图中，BCC 位于顶层控制入口，向 Vector Core、Cube、TMA 分发计算/搬运任务。
- [M] BCC 负责计算任务 (Block) 的依赖解析和调度。
- [M] Block 的输入输出是 Tile，BCC 使用 Tile 建立 Block 间依赖关系。
- [M] BCC 也包含 scalar pipe。GPR 用于不同 Block 之间的标量值交互。
- [M] 在 Vector Pipe 页中，BCC 紧邻 Vector Block，表示向量类 Block 会进入 Vector Core 内部的 TileOp 前端窗口。

7.2.2 输入输出

- 输入：来自更上层 runtime/compiler/front-end 的 Block 描述 [I]
- 输出：
 - 指向 Vector Core / Vector Block 的向量计算任务 [M]
 - 指向 Cube 的矩阵计算任务 [M]
 - 指向 TMA 的 TileReg-DDR 搬运任务 [M]

7.2.3 本文中的工作定义

本文暂把 BCC 视为：

- 块级控制器
- Tile 依赖 scoreboard / scheduler
- Vector/Cube/TMA 的任务派发源头
- 负责把 Block 级任务转发到对应执行或搬运单元
- 标量执行单元，包含 AB pipe、AM pipe 和 LSU

7.2.4 BCC Scalar Pipe

Pipe	功能	说明
AB pipe	ALU + branch	支持 ALU 与 branch，采用 2-pick 选择
AM pipe	ALU + multicycle	支持 ALU 以及多周期标量操作，例如标量乘法
LSU	scalar load/store	支持标量访存

GPR 是 BCC 标量 pipe 的寄存器基础，也可作为不同 Block 之间传递标量值的通道。

7.2.5 Block 描述符与 BCC 指令

Block 描述符以 BStart 开始。BStart 标记该段 Block 的 opcode，以及目标 PE（Vector Core、Cube 或 TMA）。一个 Block 只能发给一个目标 PE。

Block 的 Tile 输入输出字段会拆成 BCC 内部指令。典型指令如下：

指令	字段	语义
BStart	opcode, target PE, attrs	标记 Block 开始、操作类别和派发目标
B.IOT	3 src Tile + 1 dst Tile	描述输入/输出 Tile 绑定
TLOAD	memory addr, dst Tile, attrs	TMA load 类 Tile 搬运指令，需要满足内存保序

一个 B.IOT 可描述 3 src + 1 dst。多条 B.IOT 可以组合描述多输入/多输出 Block。当前外层 Block 字段约束为：dst Tile 最多 4 个，src Tile 描述字段最多 2 组；进入 BCC 后由多条 B.IOT 组合表达实际输入/输出绑定。

依赖规则：

- Block 间依赖由 Tile 输入输出建立。
- GPR 可用于 Block 间标量值交互。
- 当前不引入 flag 依赖。
- 普通计算 Block 不引入 memory side-effect 依赖。
- TMA 的 TLOAD/搬运类操作需要执行内存保序。

7.3 Vector Block 与 TileOp Window

7.3.1 图示现象

- [M] Vector Block 内部画了多行 Tsub / Texp / TcvT
- [M] 顶部写有 TileOp Issue Queue
- [M] 顶部附近标注 4KB
- [M] 右上角写有 wakeup
- [M] 右侧输出 src0 / src1 / src2

7.3.2 结构解释

本文采用以下解释：

- [I] `vector block` 本身不是单一功能框，而更像一个“TileOp 前端窗口”
- [I] `tileop issue queue` 是该窗口中的队列/阵列结构
- [I] 其中每个单元持有某类 TileOp 或 TileOp 的状态
- [I] `wakeup` 表示该窗口支持依赖解开后的重新就绪

7.3.3 功能定义

`vector block` 是 `vector core` 内部接收 BCC 向量计算任务的前端窗口，至少承担以下职责：

1. 接收来自 `bcc` 的向量类 Block/TileOp。
2. 按操作类别组织 TileOp。
3. 维护 TileOp 的依赖/就绪状态。
4. 为后级生成 `src0 / src1 / src2` 描述。
5. 在依赖满足时唤醒等待中的 TileOp。

7.3.4 关于 4KB 标注的解释

- [M] 图上 4KB 标在 `tileop issue queue` 上方
- [M] 本文将 Vector 4KB 明确解释为 Vector Tile/TileOp 源输入的基本粒度，而不是 TileReg 总容量
- [O] `tileop issue queue` 自身是否也有独立的 4KB metadata/data RAM 预算仍需和 RTL 对齐

7.4 Operand Collector

7.4.1 作用

`operand collector` 是 Janus4K 里极关键的解耦模块，其存在意味着：

- 执行前的源操作数准备被单独抽出
- 数据依赖解析不直接压在执行 Pipe 上
- “命中前递”与“发起读请求”是两条不同路径

7.4.2 输入

- `src0 / src1 / src2` [M]
- `output buffer` 查询结果 [I]

7.4.3 输出

- `dispatch` 方向的就绪 uOp [M][I]
- `read request` 到 `Read Port Arbitration` 的未命中请求 [M]

7.4.4 行为定义

本文将其行为定义为：

1. 收到一个 TileOp/uOp 的源描述。
2. 对每个源先查 `Output Buffer`。
3. 若命中，则该源可直接前递满足。
4. 若未命中，则为该源生成 `read request`。
5. 当全部源可得时，将该 uOp 送往 `uOp Issue Queue`。

这是当前图中最明确的一条“依赖优先于读数”的执行策略。

7.5 Output Buffer

7.5.1 图上明确给出的职责

图中原注释已经足够关键：

- `data forwarding` [M]
- `write-port arbitration` [M]
- `also used by column reduction, e.g. colmax` [M]

7.5.2 因此可得到的规格定义

`Output Buffer` 对应 drawio 中的 `Dst Buffer`，不能被简单理解为“写回前暂存一拍”的寄存器，而必须至少支持：

1. 以 `512 B entry` 粒度保存执行结果。
2. 依赖方查找命中。
3. 延后写回。
4. 多结果竞争同一写口时的仲裁。
5. reduction 中间结果复用。
6. 跨 Pipe forwarding。

7.5.3 建议的逻辑字段

为了支撑上述行为，正式化时 `Output Buffer` 至少需要逻辑上携带：

- 结果标签：目标 tile/目标寄存器/目标列标识 [I]
- 数据有效位 [I]
- 是否允许 forwarding 的状态位 [I]
- 是否已写回的状态位 [I]
- lock 状态位：当 TileOp 查到源数据在 `Output Buffer` 中时，该 2KB 数据窗口被锁定，直到使用完毕后才解锁 [M]
- reduction 上下文信息 [I][O]

7.5.4 设计判断

`Output Buffer` 是本设计的性能关键件之一。没有它：

- 链式依赖会大量回退到 `Tile Register File`
- 多周期结果无法被快速消费
- 写口冲突会直接堵住执行尾部
- `colmax` 这类纵向 reduction 会出现明显 bubble

7.5.5 写回与锁定规则

- 当 `TileOp` 查到源数据在 `Output Buffer` 中时，会锁定对应 `2 KB` 数据窗口。
- 被锁定的数据在 consumer 使用完毕前不得被覆盖或写回释放。
- 未被锁定的数据每拍尝试发出写回请求。
- 写回请求需要经过 `TileReg` 写口仲裁。
- reduction 中间结果可以长期保留在 `Output Buffer` 中，不必每步写回 `TileReg`。

7.6 Read Port Arbitration

7.6.1 请求来源

图中 `Read Port Arbitration` 的上游请求当前定义为三类：

- `Vector compute` [M]
- `Cube` [M]
- `TMA` [M]

其中 `Vector compute` 对应 `Operand Collector` 在 `Output Buffer` miss 后发出的通用读请求入口。

7.6.2 输出

- 发往 `Tile Register File` 的读请求 [M]
- `granted -> wakeup` [M]

7.6.3 时延约束

图中对该模块及其相邻路径明确给出：

- 正常传播时延约 `~2 cycles` [M]
- 若 routing congestion 迫使走线 detour，则可能到 `~2-3 cycles` [M]

7.6.4 规格化定义

本 AS 将其定义为：

1. 接收 `Vector compute` / `Cube` / `TMA` 三类读源请求。
2. 对 `Tile Register File` 的读端口进行仲裁。
3. 每次 grant 对应一个 `2 KB` `TileReg` 读窗口。
4. 将 grant 作为 wakeup 的一部分反馈到前端/调度侧。

7.6.5 仲裁策略

本轮确认采用 Round-Robin (RR) 仲裁：

- 三类 client 之间采用 RR 选择 grant。
- 未获 grant 的请求保持 pending，直到后续 RR 轮次被服务。
- RR 状态更新以实际 grant 为准，不能因为空队列 client 破坏有效请求的公平性。
- Vector 三源 TileOp 的 srcA/srcB/srcC 固定连续读序列仍需逐次进入仲裁；一轮完整 2 KB 三源读取通常需要 3 次 read grant。

7.7 Tile Register File / TileReg

7.7.1 基本属性

- 总容量：1 MB [M]
- Tile 粒度：每个逻辑 Tile 为 4 KB [M]
- Bank 组织：逻辑 4 bank，每个 bank 每拍读 512 B、写 512 B，bank 为 dual-port，读写带宽相同，满带宽读或写均为 2 KB/cycle [M]
- 位于页面上方中央，是读路径和执行路径的中心 [M]

7.7.2 角色定义

Tile Register File / TileReg 是 Janus4K 的核内共享数据 buffer，定位类似 DaVinci UB 或 NVIDIA Shared Memory，承担：

1. 暂存当前活跃 tile 数据。
2. 为执行流水提供源数据。
3. 接受执行结果写回。
4. 作为 Vector Core / Cube / TMA / Memory 之间的数据交换基点。

7.7.3 关于 4 KB 与 512 B 粒度的推断

- [M] 图里主要计算粒度反复出现 512 B
- [M] 一个 4 KB Tile 可拆成 8 个 512 B tile slice。Tile 内地址连续，slice 编号决定 bank 分布。
- [M] TileReg 单次满带宽读交互为 2 KB。图中示例性的三源 Vector TileOp 可连续占用 3 拍读口，分别读取 2 KB srcA、2 KB srcB、2 KB srcC，再拆成 4 个 512 B uOp/slice 迭代计算；若操作覆盖完整 4 KB Tile，则需要两个这样的 2 KB 读窗口。
- [M] TileReg 的逻辑 bank 映射保证各 PE 总能无 bank conflict 地读出某个 Tile 的 2 KB 数据窗口。

7.7.4 端口压力判断

从两页图合起来看，Tile Register File 需要同时面对：

- Vector 侧 512 B/cycle 计算供数需求 [M]
- TileReg 侧 2 KB/cycle banked read 带宽 [M]
- Cube 侧矩阵计算数据交换 [M]
- TMA 侧搬运需求 [M]

- 计算侧读请求仲裁 [M]

这说明：

- [I] 逻辑上它是多主设备共享的中心存储
- [I] 物理上几乎不可能是一个“简单单体、无 banking、无分片”的 RF
- [M] 正式版 AS 以 4 bank x 512 B/cycle dual-port bank 为基础定义读写仲裁。多个 client 同时访问时通过仲裁解决冲突。

7.8 Global Src Buffer

7.8.1 图示信息

- [M] 在 Tile Register File 右侧有一块 Src Buffer
- [M] 旁注：~6 entries; a small buffer to hide latency

7.8.2 规格定义

本文把它解释为靠近 Tile Register File 的共享读后缓冲，用于：

1. 吸收 Read Port Arbitration -> Tile Register File 的可变延迟。
2. 缓冲一次读出的 tile slice。
3. 将 RF 侧供数节奏与执行侧消耗节奏解耦。

7.8.3 深度定义

- 深度：~6 entries [M]
- entry 粒度：2 KB [M]

7.9 uOp Issue Queue

7.9.1 图示信息

- [M] 存在独立的 uOp Issue Queue
- [M] 内部分区按执行单元类别组织为 FMLA/FCVT、IALU/PERM/MAC、SFU
- [M] 左侧有 dispatch
- [M] 上方有 Data Read

7.9.2 规格定义

本文将 uOp Issue Queue 定义为：

1. 接受已通过 Operand Collector 解析的 uOp。
2. 按 opclass/执行单元类别维护等待队列。
3. 在源数据可得、目标执行单元可用时向对应 Pipe 或 SFU 发射。

7.9.3 关于队列分区的解释

本轮确认采用三类 uOp queue：

Queue	覆盖 opclass	目标执行单元	说明
FMLA/FCVT Queue	FMLA、FCVT、相关浮点转换	FMLA/FCVT Pipe	固定映射
IALU/PERM/MAC Queue	IALU、PERM、MAC、比较/归约类	IALU/PERM/MAC Pipe	固定映射
SFU Queue	EXP、DIV 等特殊函数	SFU	EXP/DIV 计算带宽 256 B/cycle

执行单元不共享；opclass 到 queue/执行单元的映射是固定映射。colmax 等 reduction 类 TileOp 可进入 IALU/PERM/MAC 类路径，但其 uOp 拆分方式需要结合输入 shape 和 element width 定义。

7.9.4 关于 Data Read 标注的解释

- [M] Data Read 位于 Tile Register File 下方与 issue 区域上方
- [I] 本文将其解释为“来自 Tile Register File 的读数据正在流入 issue/execute 区域”
- [O] 它究竟是一个独立模块名，还是单纯的连线说明，后续可校正

7.10 FMLA/FCVT Pipe

7.10.1 角色

FMLA/FCVT Pipe 是 Vector Core 中的浮点主算和格式转换执行路径，覆盖：

- FMLA 类算子
- FCVT/CVT 类算子

7.10.2 映射规则

- FMLA/FCVT Queue 固定发射到该 Pipe。
- 该 Pipe 不与 IALU/PERM/MAC Pipe 或 SFU 共享执行单元。
- 输入按 512 B slice 消费；完整 4 KB Vector TileOp 由多个 slice/uOp 组合完成。

7.11 IALU/PERM/MAC Pipe 与 SFU

7.11.1 IALU/PERM/MAC Pipe

IALU/PERM/MAC Pipe 覆盖整数、排列、MAC、比较/归约类操作：

- IALU 类算子
- PERM 类算子
- MAC 类算子
- TMAX/colmax 等比较或 reduction 类算子

IALU/PERM/MAC Queue 固定发射到该 Pipe, 该 Pipe 不共享 FMLA/FCVT 执行单元。

7.11.2 SFU

SFU 执行特殊函数类操作:

- EXP
- DIV
- 后续可扩展的特殊函数类 opclass

SFU 计算带宽为 256 B/cycle。SFU 可接收 mask Tile 作为输入; 当 SFU 操作覆盖完整 4 KB Vector Tile 时, uOp 拆分必须同时考虑 2 KB 读窗口、512 B slice 和 256 B SFU 执行节拍。

7.11.3 uOp 拆分差异

每次从 TileReg 读取源数据时仍以 2 KB 输入窗口为主, 但不同 opclass 的 uOp 拆分不同。例如 colmax 的拆分需结合列方向 reduction、输入布局 and element width, 不能简单等同于普通逐元素算子的 $4 \times 512 \text{ B}$ 计算。

7.12 Pipe-local Src Buffer / Output Buffer / Data Forwarding

7.12.1 两条 Pipe 均具备本地三级结构

在每条执行 Pipe 前都画了:

- Src Buffer
- Output Buffer
- Data Forwarding

这意味着每条 Pipe 都不是“issue 直接打进执行器”, 而是有一个小型本地 front porch。

7.12.2 本地结构的作用

本文定义如下:

- Pipe-local Src Buffer: 吸收发射抖动, 保存即将进入该 Pipe 的操作数
- Pipe-local Output path: 保存该 Pipe 最新结果, 支持局部重用、全局 Output Buffer 写入和延后回写
- Pipe-local Data Forwarding: 在本 Pipe 内部提供最近结果的短路转发

7.12.3 与全局缓冲的关系

- [M] 架构语义上的 Output Buffer 是全局唯一结构。
- [M] Pipe0/Pipe1 之间允许通过全局 Output Buffer 做跨 Pipe forwarding。
- [I] Pipe-local 标注表达靠近执行单元的入口/出口暂存和近距离转发路径; 物理实现可以把它实现为全局 Output Buffer 的分布式端口或局部旁路级, 但依赖查询语义必须收敛到全局 Output Buffer。

7.13 Cube

7.13.1 已知信息

- Cube 在 Overview 页和 Vector Pipe 页都出现 [M]
- 更新后的顶层图明确标注 Cube 是矩阵计算单元，参考 DaVinci [M]
- Cube 通过 TileReg 与其他模块交换数据 [M]

7.13.2 工作定义

本文将 Cube 定义为 Janus4K 的矩阵计算单元或张量计算单元，输入输出以 Tile 表示，由 BCC 分发任务，通过 TileReg 获取源数据并写回结果。

7.13.3 数据路径

- Cube 从 TileReg 读入数据后写入 L0A/L0B buffer。
- Cube 内部消费由脉动阵列决定，不受外部 512 B slice 计算粒度限制。
- Cube 输入/输出可以由多个 4 KB 组成的大 Tile。
- Cube 内部存在 buffer，用于吸收 TileReg 仲裁和脉动阵列消费节奏之间的差异。

7.13.4 仍需确认点

- Cube 的精确阵列规模、输入格式和输出格式
- L0A/L0B buffer 深度、banking 和与输出 buffer 的关系

7.14 TMA

7.14.1 已知信息

- TMA 在顶层图和 Vector Pipe 页中都出现 [M]
- Read Port Arbitration 接受来自 TMA 的请求 [M]
- 更新后的顶层图明确标注：TMA 负责访存，即 TileReg 和 DDR/Memory 之间交互 [M]

7.14.2 工作定义

本文将其定义为 Tile Memory Access / tile 数据搬运引擎，其作用包括：

- 在 TileReg 和 DDR/Memory 之间搬运 Tile 数据
- 在 Read Port Arbitration 中作为读请求发起方之一
- 支持 load/store 双向搬运，TileReg 读写口分离，load/store 队列共享调度资源
- 对 TMA 类 TLOAD/搬运操作执行内存保序

7.14.3 粒度与端口

- TMA 对 TileReg 的一次请求最多读或写 2 KB 数据。
- TMA 到 memory 侧的传输粒度为 256 B。
- TMA 支持读、写两个方向；读写口各自独立，调度队列共享。

7.14.4 仍需确认点

- TMA outstanding 数量、memory-side 对齐规则和异常处理

8. 数据流与执行流程

8.1 正常计算路径

基于图示和本轮确认，本设计的主流程可写成以下十一步：

1. BCC 解码 Block descriptor；每段以 BStart 开始，由 B.IOT 描述 Tile 输入输出，必要时包含 TMA TLOAD。
2. BCC 根据 Tile 输入输出关系建立依赖；不使用 flag 依赖，普通计算 Block 不建立 memory side-effect 依赖。
3. 依赖满足后，BCC 将 Block 派发给且只派发给一个目标 PE：Vector Core、Cube 或 TMA。
4. Vector 类 Block 进入 Vector Block / TileOp window，形成 src0/src1/src2、mask 和 dtype/packing 等需求。
5. Operand Collector 对每个源先查 Output Buffer。
6. 命中的源直接由前递路径满足；未命中的源生成 read request。
7. Read Port Arbitration 将来自 Vector compute、Cube、TMA 的请求汇总后按 RR 仲裁。
8. 获得 grant 的请求从 Tile Register File 读出 2 KB 数据窗口，并将 wakeup 信息反馈回来。
9. 读出数据先进入共享 Global Src Buffer，再供给 uOp Issue Queue / Pipe 本地入口。
10. uOp Issue Queue 按 FMLA/FCVT、IALU/PERM/MAC、SFU 队列向固定执行单元发射。
11. 执行结果进入全局 Output Buffer，可用于 forwarding、write-port arbitration 和 reduction reuse，之后再决定是否写回 Tile Register File。

8.2 依赖优先路径

当前图最值得保留的一点，是它明确把“依赖命中 Output Buffer”放在“去 Tile Register File 读”之前。按这个结构，依赖满足顺序应为：

1. 先看最近结果是否仍驻留在 Output Buffer。
2. 若在，则不消耗 RF 读口。
3. 若不在，再申请 RF 读。

这条规则对以下场景尤其重要：

- 链式 reduction
- 近距离 producer-consumer
- 长时延算子之后立刻被消费

8.3 TMAX 依赖示例解释

图下半部分给了一个非常明确的示意：

- max_uop0..3 先从 Tile Register 取数
- max_uop4/5 依赖前一批结果

- `max_uop6` 再依赖 `max_uop4/5`
- 中间标注 `4-cycle latency`
- 若没有 `Output Buffer`，就会出现 bubble

据此，本文把该图解释为：

- reduction 不是单拍闭环操作
- 结果至少要在 4-cycle 量级后才能被后继完全消费
- 若每一拍都必须等结果写回 `Tile Register File` 再读出，则吞吐会显著下滑
- 因此 `Output Buffer + Forwarding` 是 reduction 设计的必要条件，而不是锦上添花

8.4 Block 完成与退休

Block 在目标 PE 执行完成后，向 BCC 返回 `resolve + block_id`。`resolve` 可以乱序返回，BCC 侧按 Block 顺序退休：

- `Vector Core/Cube/TMA -> BCC` 的完成反馈至少携带 `block_id` 和 `resolve` 有效位。
- BCC 可以先记录乱序 `resolve` 状态，再按 program/block order 释放依赖和退休。
- TMA 类 `TLOAD/store` 还必须满足 TMA 内部的内存保序要求，不能仅以 Tile 依赖替代 memory ordering。

9. 调度、唤醒与发射模型

9.1 两级调度

图中同时出现 `TileOp Issue Queue` 和 `uOp Issue Queue`，这很关键。本文据此将 Janus4K 的调度组织定义为两级：

- TileOp 级：面向较粗粒度操作/块
- uOp 级：面向可发射执行的细粒度操作

9.2 外部唤醒

`Read Port Arbitration` 明确写有 `granted -> wakeup`，因此可以合理定义：

- 当源操作数相关的 RF 读请求获得 grant 并最终可读时，对应等待中的 uOp/TileOp 会被唤醒

9.3 内部唤醒

图中又单独写了 `Internal wakeup`。结合 `Output Buffer / Data Forwarding`，本文将其定义为：

- 不经过 RF 的局部结果就绪通知
- 当某个结果进入 `Output Buffer` 且可被依赖方直接消费时，等待中的依赖项可被内部唤醒

9.4 发射条件

本文给出一版最小发射条件定义：

1. uOp 已进入 `uOp Issue Queue`
2. 其所有源已 ready

- 3. 目标执行 Pipe 具备结构可用性
- 4. 必要的本地 Src Buffer 槽位可用

9.5 关于双 Pipe 的使用原则

图中直接写了：Dual pipelines improve compute-unit utilization。因此正式版 AS 至少应保留以下原则：

- 允许不同 opclass 分流到不同 Pipe
- 避免把所有算子都塞进同一条 Pipe
- 让较长时延的功能单元不阻塞所有轻量算子

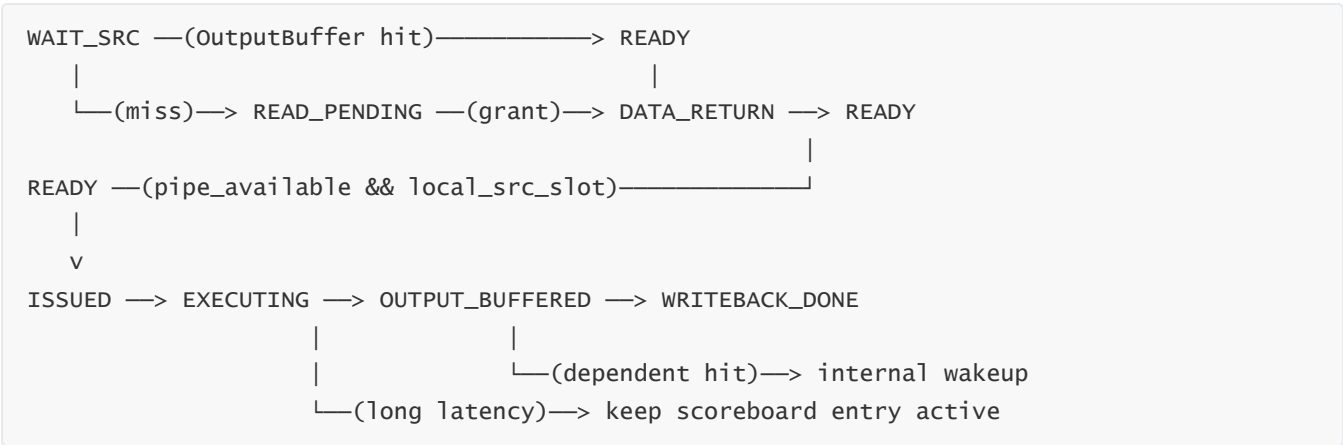
9.6 反压与流控规则

当前图没有画出完整 valid/ready，但从队列、缓冲和仲裁结构可以定义最小流控规则：

阻塞点	触发条件	上游反应	下游影响
TileOp Issue Queue 满	新 TileOp 无可入 entry	BCC 暂停派发或保持请求	不产生新的 src0/src1/src2
Operand Collector 等待源	Output Buffer miss 且读请求未 grant	uOp 暂不 dispatch	uOp Issue Queue 不增加该 uOp
Read Port Arbitration 冲突	TMA/Cube/compute 同时请求 TileReg 读口	未 grant 请求保持 pending	对应 wakeup 延后
Global Src Buffer 满	TileReg 读数据无可入缓冲 entry	仲裁器停止向 TileReg 发新读或阻塞返回	读路径形成背压
uOp Issue Queue 满	某 opclass 分区无空 entry	Operand Collector 暂停 dispatch	TileOp 可能继续等待
Pipe-local Src Buffer 满	目标 Pipe 入口无空槽	uOp 不发射	Pipe 可能出现 bubble
Output Buffer 满	结果无法驻留或写回被阻塞	执行尾部停止接收新结果	对应 Pipe 反压到 issue

最小约束是：任一缓冲满时不允许丢失 TileOp/uOp/data/tag；如果停顿会影响依赖可见性，必须优先保持 output Buffer 中的 forwarding entry 有效。

9.7 调度状态机



该状态机是调度语义模型，不要求 RTL 逐字实现同名状态，但 RTL/DV 需要覆盖这些可观察状态转换。

9.8 BCC resolve 与退休

`resolve + block_id` 是 PE 到 BCC 的完成语义。完成返回允许乱序，但退休必须顺序进行：

1. PE 完成 Block 后产生 `resolve_valid` 和 `resolve_block_id`。
2. BCC 在 reorder/retire 结构中记录该 Block 已 resolve。
3. 只有当更早 Block 均已 resolve 且满足 TMA memory ordering 时，该 Block 才能退休。
4. 退休时释放对应 Tile 依赖、GPR 依赖或后续 Block 可见状态。

10. 操作类别与执行映射

10.1 图中与本轮确认的操作类别

前端或调度区已出现 `TSUB / TEXP / TCVT / TMAX` 等 TileOp/uOp 名称；执行区和本轮确认后，Vector Core 至少按以下执行类别建模：

- `FMLA / FCVT`
- `IALU / PERM / MAC`
- `SFU`，覆盖 `EXP / DIV` 等特殊函数

10.2 固定执行映射

opclass	执行位置	说明
FMLA	FMLA/FCVT Pipe	浮点主算路径
FCVT/TCVT/CVT	FMLA/FCVT Pipe	格式转换路径
IALU/TSUB	IALU/PERM/MAC Pipe	整数/简单算术路径
PERM	IALU/PERM/MAC Pipe	排列/重排路径
MAC	IALU/PERM/MAC Pipe	MAC 类路径

opclass	执行位置	说明
TMAX/colmax	IALU/PERM/MAC Pipe	比较/归约类路径，uOp 拆分与 element width 相关
TEXP/EXP	SFU	特殊函数路径，256 B/cycle
DIV	SFU	特殊函数路径，256 B/cycle

10.3 关于不共享执行单元的说明

FMLA/FCVT、IALU/PERM/MAC、SFU 三类执行资源不共享执行单元。队列到执行单元映射固定，调度器不通过动态 steal 把一个 opclass 发射到另一类执行单元。

10.4 uOp 描述字段草案

为了让 TileOp -> uop -> Pipe 的关系可实现，建议将 uOp 至少规格化为以下字段。位宽暂以 TBD 标记，后续由 TileReg 编址、队列深度和上下文数量确定。

字段	位宽	含义
uop_id	TBD	uOp 在当前 TileOp 内的唯一编号
tileop_id	TBD	所属 TileOp 或前端窗口 entry
opclass	TBD	FMLA/FCVT/IALU/PERM/MAC/SFU/TMAX/... 等类别
src0_desc	TBD	源 0 描述，可指向 TileReg、Output Buffer 或 immediate/context
src1_desc	TBD	源 1 描述
src2_desc	TBD	源 2 描述，二源算子可置无效
mask_tile_desc	TBD	可选 mask Tile 输入
dst_desc	TBD	目标 TileReg/Output Buffer 描述
exec_class	TBD	FMLA_FCVT / IALU_PERM_MAC / SFU 固定执行类别
slice_id	TBD	当前 512 B slice 编号
tile_id	TBD	对应 4 KB Vector Tile 编号，Cube/TMA 大 Tile 可组合多个 4KB Tile
dtype_pack	TBD	数据类型和 packing，由 TileOp 的 Bstart 携带
elem_width	TBD	element width，用于 uOp 拆分和 SFU/reduction 节拍计算
read_window_id	TBD	当前 2 KB 读窗口编号
latency_class	TBD	用于 wakeup 预测和长时延跟踪
reduction_en	1	是否属于 reduction 链
forwardable	1	结果是否允许在写回前被 Output Buffer forwarding

`src*_desc` 和 `dst_desc` 必须能唯一匹配 `Output Buffer` lookup tag, 否则 forwarding 命中无法可靠判定。

11. 容量、带宽与逻辑资源假设

11.1 当前可直接引用的参数

参数	当前值	证据等级	说明
<code>Tile Register File</code> 总容量	1 MB	[M]	本轮确认
Vector Tile 容量	4 KB	[M]	Vector Tile/TileOp 源输入粒度
Cube/TMA 大 Tile	$N \times 4 \text{ KB}$	[M]	N 为正整数
TileReg 读/写窗口	2 KB	[M]	4 bank x 512B
计算 slice / Output Buffer entry	512 B	[M]	Vector 每拍计算/结果 entry 粒度
TMA memory beat	256 B	[M]	TMA 到 memory 侧传输粒度
SFU 计算带宽	256 B/cycle	[M]	EXP/DIV 等 SFU 操作
共享 Src Buffer 深度	~6 entries	[M]	向量页注释
读路径传播时延	~2 cycles	[M]	向量页注释
拥塞后读路径时延	~2-3 cycles	[M]	向量页注释
reduction 示例依赖时延	4-cycle latency	[M]	向量页示例

11.2 由参数推导出的设计含义

这些数字共同说明：

1. 数据组织粒度很粗，不是标量寄存器级。
2. 读带宽要求远高于传统简单 RF。
3. 单纯依靠 RF 读回不能支撑高利用率，必须依赖缓冲和 forwarding。
4. `Tile Register File` 是 1MB 级片上共享 working set store；其中每个 4KB Tile 是 Block 依赖和算子输入输出的基本分配单位。

11.3 建议参数化清单

参数	当前建议值	来源/说明
<code>TILE_BYTES</code>	4096	每个逻辑 Tile 为 4KB
<code>TILE_SLICE_BYTES</code>	512	Vector Core 每拍计算/传输粒度
<code>TILEREG_BYTES</code>	1 MB	TileReg 总容量
<code>TILEREG_BANKS</code>	4	顶层抽象 bank 数

参数	当前建议值	来源/说明
TILEREG_BANK_BYTES_PER_CYCLE	512	每 bank 每拍读/写带宽，读写带宽相同
TILEREG_RW_BYTES_PER_CYCLE	2048	满带宽读或写交互，4 x 512B
NUM_SRC_PER_TILEOP	3	src0/src1/src2 和三源 TileOp
MAX_DST_TILES_PER_BLOCK	4	Block 描述中的目的 Tile 上限
MAX_SRC_TILE_DESC_GROUPS_PER_BLOCK	2	Block 描述中的源 Tile 描述字段组上限
B_IOT_SRC_TILES	3	单条 B.IOT 输入 Tile 数
B_IOT_DST_TILES	1	单条 B.IOT 输出 Tile 数
NUM_EXEC_CLASSES	3	FMLA/FCVT、IALU/PERM/MAC、SFU
SFU_BYTES_PER_CYCLE	256	EXP/DIV 等特殊函数带宽
GLOBAL_SRCBUF_ENTRIES	6	图中 ~6 entries
GLOBAL_SRCBUF_ENTRY_BYTES	2048	Global Src Buffer entry 粒度
OUTPUT_BUFFER_ENTRY_BYTES	512	Output Buffer entry 粒度
TMA_REQ_BYTES	2048	TMA 对 TileReg 单次请求上限
TMA_MEM_BEAT_BYTES	256	TMA memory 侧传输粒度
READ_ARB_POLICY	RR	Vector compute/Cube/TMA 读请求仲裁
READ_PROP_LATENCY	2 cycles	图中 propagation delay
READ_CONGESTED_LATENCY	2-3 cycles	图中 congestion 注释
REDUCE_DEP_LATENCY	4 cycles	TMAX 依赖示例
OUTPUT_BUFFER_ENTRIES	TBD	需按 forwarding、writeback 和 reduction 压力确定
TILEOP_QUEUE_BYTES	4096 或 TBD	图上 TileOp Issue Queue 附近 4KB，若为独立队列容量需 RTL 对齐

这些参数应该集中进入后续 janus4k_params.py 或同等参数文件，避免 RTL、仿真模型和 AS 各自维护不同常量。

12. 时延模型与关键路径

12.1 明确时延一：RF 读路径

图上明确标出：

- ~2 cycles of propagation delay
- ~2-3 cycles if routing congestion forces longer detours

因此本文把以下链路定义为一级关键路径：

Read Port Arbitration -> Tile Register File -> Global Src Buffer

12.2 明确时延二：依赖链

TMAX 例子明确给出 4-cycle latency。本文据此认为：

- 至少某类 reduce/compare 链的结果不是单拍可用
- 调度侧必须能容忍 4-cycle 级的依赖距离
- Output Buffer 需要覆盖这种距离

12.3 对 E1 / E2 / E3 / E4 的解释

- [M] 执行区上方存在 E1 / E2 / E3 / E4 四个纵向标记
- [I] 本文将其解释为执行区的四个内部阶段边界或四个布局切片
- [O] 若后续确认它们另有含义，可整体替换

12.4 本文采用的工作时序模板

为了后续讨论方便，先给一版工作模板：

1. 周期 N：Operand Collector 解析源并查 Output Buffer
2. 周期 N+1..N+2：若 miss，则读请求经 Read Port Arbitration 到 Tile Register File
3. 周期 N+2..N+3：数据进入 Global Src Buffer
4. 周期 N+3 之后：uOp 可被发射到 Pipe 本地入口
5. 周期 N+k：执行结果进入全局 Output Buffer
6. 周期 N+k+4 左右：以 TMAX 示例为代表的后继依赖可被完全消费

这是为了讨论结构依赖而给出的时序骨架，不是最终 cycle-accurate 承诺。

12.5 Vector 4KB TileOp 读/算时序

一个完整 4 KB Vector TileOp 的标准三源读可拆成两个 2 KB 读窗口；每个窗口执行一次三源连续读取：


```
window0:
  grant/read srcA[0:2KB] -> grant/read srcB[0:2KB] -> grant/read srcC[0:2KB]
  compute 4 x 512B

window1:
  grant/read srcA[2KB:4KB] -> grant/read srcB[2KB:4KB] -> grant/read srcC[2KB:4KB]
  compute 4 x 512B
```

每个窗口通常需要 3 次 TileReg 读口 grant。普通 Vector pipe 以 512 B 为计算 slice；SFU 的 EXP/DIV 等操作以 256 B/cycle 作为执行带宽，因此同样的 2 KB 输入窗口在 SFU 中需要更细的执行节拍。

13. 写回、前递与 reduction

13.1 为什么 Output Buffer 需要优先于写回存在

若执行结果必须先写回 Tile Register File，再由依赖者重新读出，则会出现三类代价：

1. 增加 RF 写端和读端压力。
2. 拉长 producer-consumer 距离。
3. 在 reduce 链中形成额外 bubble。

因此本设计把 Output Buffer 放在写回路径之前，是合理且必要的。

13.2 写回仲裁

由于图中明确写了 write-port arbitration，本文定义：

- Output Buffer 必须允许结果在未立即获得 RF 写口时继续驻留
- 写回顺序不应破坏依赖可见性
- 依赖方可以先消费结果，再等待稍后正式落入 Tile Register File
- TileReg 写冲突通过写口仲裁解决；未被锁定的 Output Buffer entry 每拍可尝试发出写回请求

13.3 reduction 复用

图中写明 also reused by column reduction, e.g. colmax，因此：

- reduction 中间结果不应每一步都写回 RF
- Output Buffer 应允许同一 reduction 链上的部分结果持续保留
- 这类保留结果既服务下一次比较，也服务最终写回

13.4 forwarding 范围

当前图明确显示的 forwarding 是 Pipe 侧本地 forwarding，加上 Output Buffer 方向的全局命中查询。本轮确认允许跨 Pipe forwarding，因此本文采用如下定义：

- Pipe 内局部 forwarding：明确支持
- 通过全局 Output Buffer 的跨阶段 forwarding：明确支持
- Pipe0/Pipe1/SFU 之间通过全局 Output Buffer 做跨执行路径 forwarding

14. 物理实现与后端约束

14.1 读路径是物理敏感路径

图中唯一被直接用英文注释强调拥塞影响的，就是 `Tile Register File -> Src Buffer` 一带。这意味着：

- 此路径对布线距离敏感
- 仲裁和 RF 本体不宜相距过远
- `Src Buffer` 应靠近主要消费者或 RF 出口

14.2 建议的相对布局

基于现有图的布局逻辑，推荐保持以下相对位置关系：

1. `Read Port Arbitration` 紧邻 `Tile Register File`。
2. `Global Src Buffer` 放在 `Tile Register File` 的执行侧出口。
3. 两条主要 `Execute Pipe` 和 `SFU` 水平展开，Pipe 本地 `Src/Output/Forward` 紧贴 Pipe 入口/出口。
4. 全局 `Output Buffer` 靠近 `Operand Collector / dispatch` 区域，便于依赖查询。

14.3 布线/拥塞风险

如果该设计走到实现阶段，最可能出现的后端问题是：

- `Tile Register File` 出口扇出过大
- 读口仲裁控制线与数据线交织，导致拥塞
- 两条 Pipe 拉得过长，导致共享结构到 Pipe 的连接过远
- 为了满足所有口数而过度多端口化，导致面积、时序和功耗失控

14.4 规避方向

推荐后续在正式架构版补上以下内容：

- `Tile Register File` banking 方案
- 共享 `Src Buffer` 的 entry 粒度与出队规则
- `Output Buffer` 的 tag 组织
- 读/写冲突处理的精确定义

15. 性能意图与潜在瓶颈

15.1 双 Pipe 的真实目标

图已经直接点明，双 Pipe 的核心目标是 `improve compute-unit utilization`。这说明设计者关注的是实际 sustained throughput，而不是只看峰值。

15.2 预计瓶颈

按当前图示，最可能成为瓶颈的点包括：

- 1. Read Port Arbitration 的冲突与公平性。
- 2. Tile Register File 的端口数与物理实现复杂度。
- 3. Global Src Buffer 深度是否足够吸收 2-3 cycle 波动。
- 4. Output Buffer 是否足够强，能否覆盖 reduction、跨 Pipe forwarding 和延后写回。
- 5. FMLA/FCVT、IALU/PERM/MAC、SFU 三类固定执行资源的负载平衡是否合理。

15.3 关于“可能浪费”的架构含义

顶层图里“可能浪费”这几个字非常值得保留。它实际提醒了两个问题：

- 静态分配出来的端口未必都能被 workload 吃满
- 不同数据路径之间可能需要一定的共享/复用策略，否则会出现一边拥塞、一边闲置

16. 接口定义草案

图中没有给出真实 RTL 信号名和位宽，因此本节先定义逻辑接口。正式 RTL 命名可以不同，但必须保留等价语义、方向、事务身份和反压关系。

16.1 握手约定

除非单独说明，本文接口默认采用 valid/ready 握手：

```
transfer = valid & ready
```

- valid 由发送方驱动，表示 payload 有效。
- ready 由接收方驱动，表示本拍可接收 payload。
- 当 valid=1 且 ready=0 时，发送方必须保持 payload 稳定。
- 任一事务跨多个阶段流动时，必须携带 block_id/tileop_id/uop_id/tag 中至少一种身份字段。

16.2 BCC -> Vector Core / Cube / TMA

信号	位宽	方向	含义
block_valid	1	output	BCC 派发 Block 有效
block_ready	1	input	目标单元可接收 Block
block_target	TBD	output	vector core / cube / TMA
block_id	TBD	output	Block 唯一编号
block_opcode	TBD	output	Block 操作类别

信号	位宽	方向	含义
<code>block_bstart</code>	TBD	output	<code>Bstart</code> 解码结果, 携带 opcode、target PE、dtype/packing、attrs
<code>block_iot_vec</code>	TBD	output	一条或多条 <code>B.IOT</code> 展开后的 Tile 输入输出绑定
<code>block_src_tile_desc</code>	TBD	output	源 Tile 描述字段组, 当前最多 2 组
<code>block_dst_tile_vec</code>	TBD	output	目的 Tile 列表, 当前最多 4 个
<code>block_mask_tile</code>	TBD	output	可选 mask Tile
<code>block_tload_desc</code>	TBD	output	TMA <code>TLOAD</code> /搬运描述, 非 TMA Block 可无效
<code>block_attr</code>	TBD	output	reduction、memory ordering、element width 等属性

`BCC` 使用 Tile 输入输出关系建立 Block 依赖。只有当依赖满足且唯一目标单元 `ready` 时, Block 才能被派发; 普通计算 Block 不使用 flag 依赖或 memory side-effect 依赖, TMA `TLOAD` /store 需要遵守内存保序。

16.2.1 Vector Core / Cube / TMA -> BCC resolve

信号	位宽	方向	含义
<code>resolve_valid</code>	1	input	PE 完成 Block 并返回 resolve
<code>resolve_ready</code>	1	output	BCC 可接收完成反馈
<code>resolve_block_id</code>	TBD	input	已完成的 Block ID
<code>resolve_status</code>	TBD	input	成功/异常/错误状态

`resolve` 可以乱序返回; BCC 必须按 Block 顺序退休, 并在退休时释放 Tile/GPR 依赖。

16.3 Vector Block -> operand collector

信号	位宽	方向	含义
<code>oc_valid</code>	1	output	源描述/uOp 描述有效
<code>oc_ready</code>	1	input	Operand Collector 可接收
<code>oc_tileop_id</code>	TBD	output	对应 TileOp entry
<code>oc_uop_id</code>	TBD	output	对应 uOp 编号
<code>oc_opclass</code>	TBD	output	<code>FMLA/FCVT/IALU/PERM/MAC/SFU/TMAX/...</code>
<code>oc_src0_desc</code>	TBD	output	源 0 描述
<code>oc_src1_desc</code>	TBD	output	源 1 描述

信号	位宽	方向	含义
oc_src2_desc	TBD	output	源 2 描述
oc_mask_tile_desc	TBD	output	可选 mask Tile 描述
oc_dst_desc	TBD	output	目的描述
oc_slice_id	TBD	output	当前 512 B slice 编号
oc_read_window_id	TBD	output	当前 2 KB 读窗口编号
oc_dtype_pack	TBD	output	来自 BStart 的 dtype/packing

Operand Collector 接收后必须先执行 `Output Buffer` lookup, 再决定是否发 TileReg read request。

16.4 Operand collector -> Read Port Arbitration

信号	位宽	方向	含义
rd_req_valid	1	output	TileReg 读请求有效
rd_req_ready	1	input	仲裁器可接收请求
rd_req_tile_id	TBD	output	目标 Tile 编号
rd_req_slice_base	TBD	output	目标 512 B slice 或 2 KB 读窗口
rd_req_bytes	TBD	output	请求字节数, 典型为 2 KB
rd_req_tag	TBD	output	返回数据和 wakeup 匹配 tag
rd_req_src_idx	2	output	src0/src1/src2 编号
rd_req_client	TBD	output	请求来源: compute/TMA/Cube

同一个 Vector TileOp 典型情况下可产生 3 个源读请求, 分别读取 `srcA/srcB/srcC`。若某个源被 `Output Buffer` 命中, 则不应再消耗 TileReg 读口。仲裁策略固定为 `RR`。

16.5 Read Port Arbitration -> TileReg

信号	位宽	方向	含义
tr_rd_valid	1	output	读请求已仲裁通过
tr_rd_ready	1	input	TileReg 可接收读请求
tr_rd_tile_id	TBD	output	Tile 编号
tr_rd_bank_mask	4	output	参与读的 bank mask
tr_rd_addr	TBD	output	TileReg 内部地址或 slice 编号

信号	位宽	方向	含义
tr_rd_bytes	TBD	output	读字节数，满带宽为 2 KB
tr_rd_tag	TBD	output	原样随读返回
tr_rd_client	TBD	output	请求来源，用于统计和调试

该路径是物理敏感路径。AS 默认目标为正常情况下约 2 cycles 到达数据返回路径；拥塞或绕线情况下可放宽到 2-3 cycles，但必须由性能模型吸收。

16.6 TileReg -> Global Src Buffer

信号	位宽	方向	含义
srcbuf_valid	1	output	TileReg 返回数据有效
srcbuf_ready	1	input	Global Src Buffer 可接收
srcbuf_tag	TBD	output	与读请求 tag 匹配
srcbuf_src_idx	2	output	对应源编号
srcbuf_data	16384	output	2 KB 读窗口数据
srcbuf_err	TBD	output	可选错误状态

srcbuf_data 暂按 2 KB = 16384-bit 定义。进入执行区时可进一步拆成 4 个 512 B slice。

16.7 Operand collector / Src Buffer -> uOp Issue Queue

信号	位宽	方向	含义
uop_valid	1	output	uOp 已满足最小入队条件
uop_ready	1	input	uOp Issue Queue 可接收
uop_payload	TBD	output	第 10.4 节定义的 uOp 描述
uop_src_ready_mask	3	output	三个源是否已 ready
uop_forward_hit_mask	3	output	三个源是否来自 Output Buffer forwarding

当 uop_src_ready_mask 未全 1 时，该 uOp 不允许进入可发射状态；是否允许先入队等待数据由后续实现选择，但 AS 必须明确区分 queued 和 ready-to-issue。

16.8 uOp Issue Queue -> Execute Pipe/SFU

信号	位宽	方向	含义
pipe_issue_valid	1	output	发射到目标执行单元有效
pipe_issue_ready	1	input	Pipe-local Src Buffer 或 SFU 输入缓冲可接收
exec_class	TBD	output	FMLA_FCVT / IALU_PERM_MAC / SFU
pipe_uop	TBD	output	uOp payload
pipe_src0_data	4096	output	源 0 的 512 B slice
pipe_src1_data	4096	output	源 1 的 512 B slice
pipe_src2_data	4096	output	源 2 的 512 B slice, 可选

对于不需要三源 512B 的轻量算子，数据总线可被实现为复用或分段，但 AS 逻辑上仍以三源模型描述。

16.9 Execute Pipe/SFU -> Output Buffer

信号	位宽	方向	含义
ob_valid	1	output	执行结果有效
ob_ready	1	input	Output Buffer 可接收
ob_tag	TBD	output	forwarding/writeback 匹配 tag
ob_data	4096	output	512 B 结果 entry
ob_forwardable	1	output	结果是否可前递
ob_writeback_req	1	output	结果最终是否需要写回 TileReg
ob_reduction_ctx	TBD	output	reduction 链上下文

若 ob_ready=0，Pipe/SFU 必须保持结果或停止新发射，不能丢失 forwarding-visible 结果。

16.10 Output Buffer -> TileReg 写回

信号	位宽	方向	含义
tr_wr_valid	1	output	写回请求有效
tr_wr_ready	1	input	TileReg 写口可接收
tr_wr_tile_id	TBD	output	写回目标 Tile
tr_wr_slice_id	TBD	output	写回目标 512 B slice
tr_wr_data	4096	output	512 B 写回数据

信号	位宽	方向	含义
tr_wr_mask	TBD	output	字节/片段写 mask，支持部分写时使用
tr_wr_tag	TBD	output	调试/完成跟踪 tag
tr_wr_lock	1	internal	对应 2 KB 数据窗口被 consumer 锁定时禁止释放/覆盖

写回仲裁不能破坏 forwarding 语义：结果可以先被依赖方消费，再在稍后写回 TileReg。未被锁定的 Output Buffer entry 每拍尝试发出写回请求，TileReg 写冲突通过写口仲裁解决。

16.11 TMA <-> TileReg / Memory

信号	位宽	方向	含义
tma_cmd_valid	1	input	TMA 命令有效
tma_cmd_ready	1	output	TMA 可接收命令
tma_cmd_is_store	1	input	0=load/TLOAD, 1=store
tma_tile_id	TBD	input	TileReg 目标或源 Tile
tma_tile_bytes	TBD	input	单次 TileReg 请求字节数，最大 2 KB
tma_mem_addr	TBD	input	memory 地址
tma_mem_beat_bytes	TBD	input	memory 侧 beat，固定 256 B
tma_order_tag	TBD	input	内存保序 tag

TMA 读写口分离、队列共享。load/store 都必须以 2 KB TileReg 请求为上限，并在 memory 侧拆成 256 B beat。

17. 实现代码结构规划

当前仓库中未看到 Janus4K 的 RTL/pyCircuit 实现文件，因此本节先给出建议文件结构，用于后续从 AS 落到模型或 RTL。

17.1 建议目录结构

```
janus/pyc/janus/janus4k/
├── __init__.py
├── janus4k_core.py           # 顶层连接: BCC/Vector/Cube/TMA/TileReg
├── janus4k_params.py        # 第 11.3 节参数集中定义
├── bcc.py                   # Block Control Core
├── block_decoder.py         # BStart / B.IOT / TLOAD descriptor 解码
├── gpr.py                   # BCC GPR
├── bcc_scalar_pipe.py       # BCC scalar pipe 顶层
├── ab_pipe.py               # ALU + branch, 2-pick
├── am_pipe.py               # ALU + multicycle / scalar multiply
├── bcc_lsu.py               # scalar load/store
├── retire.py                # resolve 乱序记录与顺序退休
```


└─ tileop_window.py	# TileOp Issue Queue + wakeup
└─ operand_collector.py	# Output Buffer lookup + read request generation
└─ read_port_arb.py	# TMA/Cube/compute RR 读请求仲裁
└─ tile_register_file.py	# 1MB TileReg / 4-bank read path
└─ src_buffer.py	# Global Src Buffer
└─ uop_issue_queue.py	# opclass 分区队列与 Pipe 发射
└─ execute_pipe.py	# FMLA/FCVT 与 IALU/PERM/MAC 公共框架
└─ sfu.py	# EXP/DIV 等 256B/cycle 特殊函数单元
└─ output_buffer.py	# forwarding / writeback arbitration / reduction reuse
└─ tma.py	# TileReg-DDR 搬运
└─ cube_if.py	# Cube 接口、LOA/LOB buffer 占位
└─ janus4k_debug.py	# trace/debug signal bundle

17.2 模块到文件映射

AS 模块	建议文件	关键职责
BCC	bcc.py / block_decoder.py / retire.py	Block 依赖解析、descriptor 解码、调度、目标单元派发和顺序退休
BCC Scalar Pipe	gpr.py / bcc_scalar_pipe.py / ab_pipe.py / am_pipe.py / bcc_lsu.py	GPR、AB/AM/LSU 标量执行和 Block 间标量值交互
Vector Block / TileOp Issue Queue	tileop_window.py	TileOp 入队、状态跟踪、外部/内部 wakeup
Operand Collector	operand_collector.py	三源解析、Output Buffer lookup、TileReg read request
Read Port Arbitration	read_port_arb.py	TMA/Cube/compute 读请求 RR 仲裁和 grant
TileReg	tile_register_file.py	1MB 存储、4-bank 读、写回端口和冲突规则
Global Src Buffer	src_buffer.py	约 6 entry、每 entry 2KB 的读后缓冲、tag 匹配
uOp Issue Queue	uop_issue_queue.py	opclass 分区、ready 选择、Pipe 发射
FMLA/FCVT Pipe	execute_pipe.py	FMLA/FCVT 执行路径
IALU/PERM/MAC Pipe	execute_pipe.py	IALU/PERM/MAC/compare/reduction 执行路径
SFU	sfu.py	EXP/DIV 等 256B/cycle 特殊函数
Output Buffer	output_buffer.py	forwarding、write-port arbitration、col reduction reuse、2KB lock
TMA	tma.py	TileReg 与 DDR/Memory 之间双向搬运、2KB TileReg 请求、256B memory beat

17.3 调试信号建议

信号	含义
dbg_block_valid_vec	BCC 已派发或等待派发的 Block valid
dbg_tileopq_valid_vec	TileOp Issue Queue 各 entry valid
dbg_uopq_valid_vec	uOp Issue Queue 各 opclass entry valid
dbg_rd_arb_grant_client	本拍 TileReg 读口 grant 给 compute/TMA/Cube 哪一类 client
dbg_rd_arb_rr_ptr	RR 仲裁指针
dbg_srcbuf_occupancy	Global Src Buffer 占用度
dbg_outputbuf_occupancy	Output Buffer 占用度
dbg_outputbuf_lock_vec	Output Buffer 2KB 窗口 lock 状态
dbg_forward_hit_mask	本拍 forwarding 命中的源 mask
dbg_fm1a_busy/dbg_ialu_busy/dbg_sfu_busy	三类执行资源的忙闲状态
dbg_resolve_pending_vec	BCC 已 resolve 但尚未退休的 Block
dbg_writeback_stall	写回口阻塞导致的 Output Buffer 停顿

18. 测试验证

为了让这份 AS 后面能转成更正式的验证输入，建议至少覆盖以下测试。

18.1 基础测试用例

Test 1: BCC Block 派发

- 构造 Vector/Cube/TMA 三类 Block。
 - 每个 Block 以 Bstart 开始，并用 B.IOT 描述 Tile 输入输出。
 - 建立 Tile 输入输出依赖关系。
 - 验证 BCC 只在依赖满足时派发。
 - 验证每个 Block 只发给一个 target PE。
 - 验证 block_target 正确选择 Vector Core、Cube 或 TMA。

Test 2: BCC Scalar Pipe / GPR

- 在 Block A 中通过 AB/AM pipe 产生一个标量值并写入 GPR。
 - 在 Block B 中读取该 GPR 值作为标量输入。
 - 覆盖 AB pipe ALU+branch 2-pick、AM pipe 标量乘法和 LSU 标量访存。
 - 验证 GPR 可用于不同 Block 间标量值交互。

Test 3: Vector 三源 TileOp 标准路径

1. BCC 派发一个三源 Vector TileOp。
2. Operand Collector 对三个源全部 miss Output Buffer。
3. Read Port Arbitration 依次 grant srcA/srcB/srcC 三个 TileReg read request。
4. 每个源读出 2KB 窗口，并拆成 4 个 512B uOp/slice。
5. 完整 4KB TileOp 覆盖两轮 2KB 三源读取。
6. uOp Issue Queue 发射到固定执行单元。
7. 结果进入 Output Buffer 并按 512B entry 写回 TileReg。
8. 验证 output 数据、tag、tile_id 和 slice_id 匹配。

Test 4: Output Buffer forwarding hit / lock

1. 先执行 producer uOp，使结果停留在 Output Buffer。
2. 派发 dependent uOp，其 src0 命中 Output Buffer。
3. 验证该源不产生 TileReg read request。
4. 验证 dependent uOp 可通过 internal wakeup 进入 ready 状态。
5. 验证命中的 2KB 数据窗口被 lock，consumer 使用完毕后 unlock。
6. 验证未被 lock 的 entry 每拍尝试发出写回请求，并通过 TileReg 写仲裁。

Test 5: TMA/Cube/compute 读口 RR 争用

1. 同一窗口内同时注入 TMA、Cube 和 compute 读请求。
2. 验证仲裁器不会丢请求。
3. 验证 grant 顺序符合 RR 策略。
4. 验证 pending 请求在后续周期获得 grant。

Test 6: TMAX/colmax 4-cycle reduction 依赖链

1. 构造 max_uop0..6 的树形依赖。
2. 设置 producer-consumer latency = 4 cycles。
3. 打开 Output Buffer forwarding。
4. 验证 max_uop4/5/6 不需要等待中间结果先写回 TileReg。
5. 验证 reduction 中间结果可长期驻留 Output Buffer。

Test 7: TMA load/store 粒度与保序

1. 构造 TLOAD 和 store 混合序列。
2. 验证每次 TMA TileReg 请求不超过 2KB。
3. 验证 memory 侧拆成 256B beat。
4. 验证 load/store 共享队列、读写口分离。
5. 验证 TMA 内存保序规则。

Test 8: SFU 256B exp/div

1. 构造 EXP/DIV TileOp，携带 dtype/packing 和可选 mask Tile。
2. 验证 SFU 发射来自 SFU Queue。
3. 验证 SFU 以 256B/cycle 消费数据。
4. 验证 mask Tile 生效且不破坏 Output Buffer tag。

Test 9: BCC resolve OOO / in-order retire

1. 连续派发多个 Block 到不同 PE。
2. 让后发 Block 先返回 resolve + block_id。
3. 验证 BCC 能记录乱序 resolve。
4. 验证最终退休仍严格按 Block 顺序。

18.2 性能/压力场景

1. Output Buffer 命中与 miss 混合出现。
2. 拥塞导致读路径从 2 cycle 拉长到 3 cycle。
3. FMLA/FCVT、IALU/PERM/MAC、SFU 同时繁忙。
4. 某一固定执行资源繁忙而其他资源空闲，验证不共享执行单元时的调度行为。
5. reduction 中间结果长时间驻留 Output Buffer。
6. Global Src Buffer 接近满时持续返回 TileReg read data。
7. 写回口被阻塞时，执行尾部和 Output Buffer 的反压传播。
8. BCC 连续派发 Vector/Cube/TMA 混合 Block，检查 Tile 依赖和 TileReg 仲裁。
9. TileReg 连续 2KB 读窗口跨逻辑 bank 访问，验证无 bank conflict。

18.3 验证要点

- block_id/tileop_id/uop_id/tag 在读请求、数据返回、wakeup、执行、写回链路中保持一致。
- Output Buffer hit 时不消耗 TileReg 读口。
- Output Buffer 满时不丢失 forwarding-visible 结果。
- Read Port Arbitration 对 compute/TMA/Cube 请求执行 RR。
- Global Src Buffer 满时反压路径明确。
- opclass -> Pipe/SFU 固定映射与第 10.2 节一致。
- TMAX 依赖链在 4-cycle latency 下不会产生不必要 bubble。
- resolve 允许乱序返回，但 Block 退休必须顺序。

19. 开放问题与下一轮校正清单

这部分只保留本轮问答后仍未完全定死的点。

19.1 命名与职责

- cube 的精确阵列规模、输入输出 layout 和 TileReg 端口需求是什么？
- TMA outstanding 数量、memory-side 对齐规则和异常/错误上报如何定义？

19.2 存储组织

- TileReg 的 1MB 地址空间如何编码到 logical bank、512B slice 和 Tile ID？
- Tileop Issue Queue 上方的 4KB 是否代表独立 metadata/data RAM 容量？
- Output Buffer 深度、tag CAM 组织、2KB lock 粒度和 512B entry 的映射关系如何实现？
- Global Src Buffer 的实际 entry 数是否固定为 6，还是可参数化？

19.3 调度与发射

- 不同 opclass、element width、mask Tile 下，TileOp 到 uOp 的拆分公式是什么？
- FMLA/FCVT、IALU/PERM/MAC、SFU 三类 queue 的深度、选择策略和 backpressure 规则是什么？
- resolve 乱序返回后，BCC retire buffer 深度和异常处理规则是什么？

19.4 前递与写回

- Output Buffer 中结果何时必须写回 Tile Register File，何时可以只被 consumer 消费后释放？
- cross-pipe forwarding 的物理路径是全局 CAM/数据通路，还是局部端口接入全局 Output Buffer？

19.5 顶层互联

- BStart/B.IOT/TLOAD 的实际编码位宽和压缩格式是什么？
- mask Tile 的 layout、有效粒度和与 dtype/packing 的关系是什么？
- Cube 大 Tile 与 Vector 4KB Tile 的 Tile ID/地址映射关系如何统一？

20. 结论

在当前信息不完整的前提下，这张图已经足够说明 Janus4K 的核心设计哲学：

1. 它是一个围绕 TileReg 组织的数据中心型 AI 核子系统，TileReg 是 1MB 级共享数据 buffer。
2. 它采用 Block -> TileOp -> uOp/slice 的层次组织：BCC 管 Block 依赖和派发，Vector Core 管 TileOp/uOp 执行。
3. 它不把所有依赖都压回 TileReg，而是用 Output Buffer + Data Forwarding 解决近距离结果重用。
4. 它明确承认物理实现会反过来影响微架构，尤其是 TileReg 读路径的拥塞和走线时延。
5. 它引入双执行流水，目标是提升真实利用率而不是追求形式上的模块堆叠。

如果把这份草案继续收敛成正式评审版，优先级最高的是把以下四件事钉死：

- TileReg 的地址编码和 logical bank/slice 映射
- Output Buffer 的深度、tag、2KB lock 和写回释放规则
- TileOp -> uOp 在不同 opclass/element width/mask 下的拆分公式
- Cube/TMA 大 Tile 与 Vector 4KB Tile 的统一编址和接口规则

附录 A: 术语表

术语	含义
Janus4K	当前图示中的 TileReg-centered AI core 子系统
BCC	Block Control Core，负责 Block 依赖解析、调度、任务分发、标量 pipe 和顺序退休
GPR	BCC 内部通用寄存器文件，用于不同 Block 间标量值交互
AB pipe	BCC scalar pipe 中的 ALU + branch 管线，采用 2-pick

术语	含义
AM pipe	BCC scalar pipe 中的 ALU + multicycle 管线，支持标量乘法
BCC LSU	BCC scalar pipe 中的标量 load/store 单元
Block	BCC 看到的任务粒度，输入输出用 Tile 表示
BStart	Block descriptor 起始指令，携带 opcode、target PE、dtype/packing 等属性
B.IOT	BCC Tile 输入输出绑定指令，单条描述 3 src + 1 dst
TLOAD	TMA load 类 Tile 搬运指令，需要满足内存保序
Tile	Vector 侧 4 KB 逻辑数据块；Cube/TMA 可使用 4KB 整数倍的大 Tile
Tile slice	512 B 计算/传输片段，是 Vector Core 每拍处理粒度
TileReg	AI 核内共享数据 buffer，总容量 1 MB
TileOp	Vector Core 内部的 tile 级向量操作
uOp	可被 uOp Issue Queue 独立调度到执行 Pipe 的细粒度操作
Operand Collector	源操作数收集与 Output Buffer lookup 模块
Output Buffer	drawio 中 Dst Buffer 的正式架构名，负责结果驻留、forwarding、write-port arbitration 和 reduction reuse
Global Src Buffer	TileReg 读出后的共享小缓冲，图示约 6 entries，每 entry 2 KB
Read Port Arbitration	compute/TMA/Cube 读请求到 TileReg 的 RR 仲裁模块
Pipe-local Src Buffer	执行 Pipe 入口附近的局部源缓冲
Pipe-local Output path	执行 Pipe 尾部附近的局部结果/旁路路径
Forwarding	结果未写回 TileReg 前被依赖方直接消费
Wakeup	源数据或依赖结果就绪后唤醒等待中的 TileOp/uOp
Resolve	PE 执行完 Block 后向 BCC 返回的完成反馈，携带 block_id
Cube	矩阵计算单元，参考 DaVinci 风格 Cube，内部有 L0A/L0B buffer
TMA	Tile Memory Access，负责 TileReg 与 DDR/Memory 之间双向搬运
Mask Tile	作为 TileOp 可选输入的 mask 数据
SFU	Special Function Unit，执行 EXP/DIV 等 256 B/cycle 特殊函数
TMAX	图中示例 reduction/compare 类 TileOp/uOp

术语	含义
TSUB	图中示例 subtract 类 TileOp/uOp
TEXP	图中示例 exp 类 TileOp/uOp
TCVT	图中示例 convert 类 TileOp/uOp

附录 B: 图示证据索引

图示文字	本文使用位置	解释
Block Control Core	第 6 / 7 / 16 节	BCC 全称和职责
负责计算任务（Block）的解依赖和调度	第 6 / 7 / 18 节	BCC 负责 Block 依赖解析和调度
Block的输入输出是Tile	第 5 / 6 / 16 节	Tile 是 Block 依赖建立粒度
向量执行单元，计算带宽是512B	第 5 / 6 / 11 节	Vector Core 每拍 512B 计算带宽
矩阵计算单元，参考Davinci	第 7 / 16 节	Cube 定义
访存，TileReg和DDR之间交互	第 7 / 16 节	TMA 定义
TileRegisterfile 4K	第 7 / 11 节	Vector Pipe 图中的当前工作 Tile 粒度
4KB 注释	第 5 / 7 / 11 节	每个 Tile 为 4KB，TileOp queue 标注仍待确认
ReadPortArbitration	第 7 / 12 / 16 节	TileReg 读请求仲裁
granted -> wakeup	第 9 / 16 节	TileReg read grant 触发等待项唤醒
~ 2 cycles propagation delay	第 12 / 16 节	正常读路径延迟模型
~2-3 cycles if routing congestion...	第 12 / 14 / 18 节	拥塞后的延迟放宽和验证场景
~6 entries; small buffer	第 7 / 11 / 17 节	Global Src Buffer 深度
data forwarding and write-port arbitration	第 7 / 13 / 16 节	Dst Buffer 核心职责
also used by col reduce, colmax	第 7 / 13 / 18 节	reduction reuse 需求

图示文字	本文使用位置	解释
Dual pipelines improve compute-unit utilization	第 9 / 15 / 18 节	双 Pipe 的性能目标
4-cycle latency	第 8 / 12 / 18 节	TMAX 依赖链测试约束